



AI with Rav

Learn AI the way you actually think.



DAY 5 of 30

PYTHON

Input, Output & Comments — a two-way conversation

WHAT YOU'LL LEARN TODAY

- › `print()` speaks to the user
- › `input()` listens back
- › The big gotcha: input is always text
- › Showing several things at once
- › Comments — notes for humans



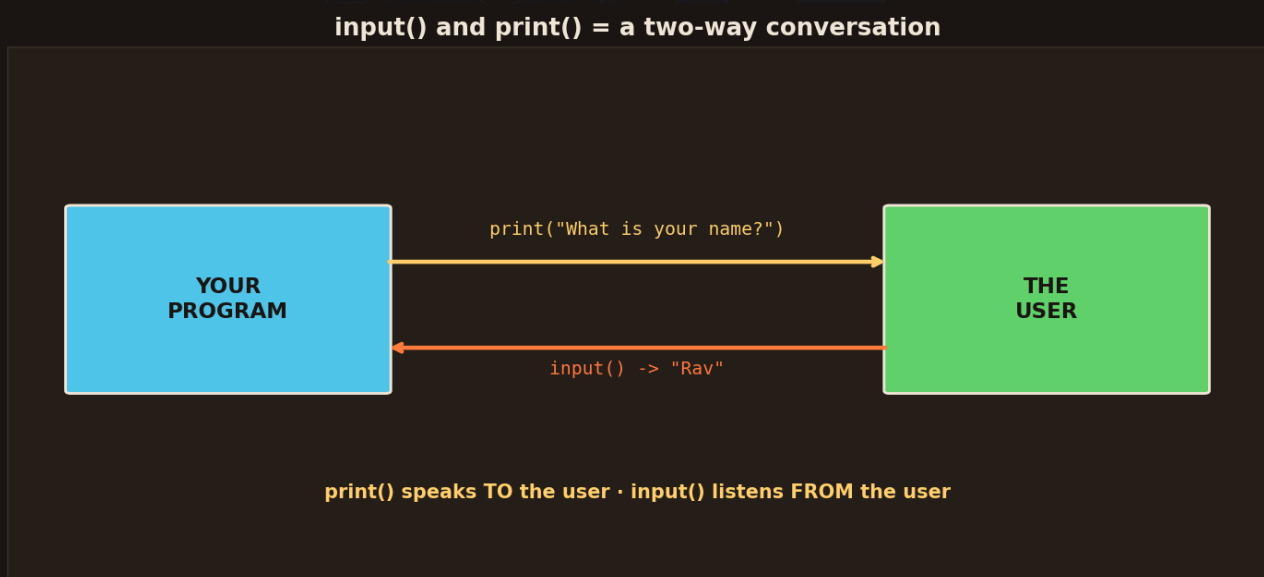
Input, Output & Comments — a two-way conversation

In One Line: Until now your program only *spoke* (print). Today it also *listens* — you ask the user a question with `input()`, and use their answer. That turns a script into a real conversation. Plus: comments (`#`) — little notes you leave for humans.

Start here: make it two-way

So far your programs have been a one-way street — Python *tells* you things with ``print()``. But a real program is a **conversation**: it asks the user something, waits, and does something with the reply. A calculator asks for numbers, a signup asks for a name. Today we add the missing half — **listening** — with ``input()``.

print() and input() — a two-way conversation



`input()` and `print()` are a two-way conversation. `print("What is your name?")` speaks TO the user (yellow arrow out). `input()` listens FROM the user and hands back what they typed (orange arrow back). `print` speaks, `input` listens.

Think of your program talking to a person:

- ``print(...)`` → your program **speaks** to the user (shows text on screen).
- ``input(...)`` → your program **listens** — it pauses, waits for the user to type, and hands back whatever they typed.

```
name = input("What is your name? ")
print("Hello, " + name + "!")
```



``input("What is your name? ")`` shows the question, then **waits**. The moment the user types something and presses Enter, that text is handed back and stored in ``name``. The text inside the brackets is just the prompt — the question you want to ask.

The big gotcha: input is ALWAYS text

Careful: input() ALWAYS gives back text (a string)

```
age = input("Your age? ")    # user types 30
```

"30"

-> so age + 1 would ERROR

text, NOT a number!

```
age = int(input("Your age? "))    # NOW it is a number
```

Wrap it in int() (or float()) to convert text to a number

`input()` ALWAYS gives back text, even if the user types 30 — it becomes the string "30", not the number 30. So `age + 1` would error. Fix: wrap it as `int(input(...))` to convert the text into a real number.

This is the #1 beginner surprise, so read it twice: **whatever ``input()`` gives back is always text (a string)** — even if the user types digits.

```
age = input("Your age? ")    # user types 30
print(age + 1)               # ERROR! age is the TEXT "30", not the number 30
```

The fix is to **convert** the text into a real number by wrapping it in ``int()``:

```
age = int(input("Your age? "))    # now age is the number 30
print(age + 1)                   # -> 31
```

Burn this in: **input() never gives you a number — it gives you text**. If you need to do maths with it, wrap it: ``int(...)`` for whole numbers, ``float(...)`` for decimals. Forgetting this causes the classic "can only add str to str" error. We go deep on converting types on Day 8 — for now, just remember the ``int(input(...))`` wrap.

print() can show several things at once



print() can show several things at once

```
print("Age:", 30, "years")
```

Age: 30 years

Separate items with commas — print adds a space between them

`print("Age:", 30, "years")` shows Age: 30 years. Separate items with commas — print automatically puts a space between each one.

You don't need ``+`` to show several things. Just separate them with **commas** — and ``print`` mixes text and numbers for you, adding a space between each:

```
print("Age:", 30, "years")      # -> Age: 30 years
name = "Rav"
print("Hi", name, "welcome!")  # -> Hi Rav welcome!
```

Commas are the easy way: ``print("Age:", 30)`` just works — no need to convert the ``30`` to text, and Python adds the spaces. Compare that with ``+``, which needs everything to be text and no automatic spaces. **(If you already code:** commas here are like passing multiple arguments — ``print`` handles the joining.) For fancier messages, the f-strings from Day 3 are even nicer.

Comments — notes for humans

Comments (#): notes for humans, ignored by Python

```
# work out the total price
price = 100
tax = 18          # 18% GST
total = price + tax # add them up
print(total)
```

Anything after # is a note. Explain the WHY, not the obvious.



Comments start with `#`. Anything after `#` on a line is a note Python ignores — it is only for humans reading the code.

Example: `tax = 18 # 18% GST`. Explain the **WHY**, not the obvious.

A **comment** starts with ``#``. Python completely ignores it — it's a note written for **humans** who read the code later (usually your future self):

```
# work out the final bill
price = 100
tax = 18          # 18% GST
total = price + tax
print(total)      # -> 118
```

Two ways to use ``#``: on its own line (a heading for a section) or at the end of a line (a quick note). The golden rule: **explain the WHY, not the obvious**. ``total = price + tax # add them up`` is useless — we can see that. ``tax = 18 # 18% GST`` is useful — it explains *why* 18. Good comments make you look like a pro.

Putting it together — a tiny interactive program

Putting it together: a tiny interactive program

```
name = input("Your name? ")
age  = int(input("Your age? "))
next_age = age + 1
print(f"Hi {name}, next year you turn {next_age}")
```

Hi Rav, next year you turn 31

A small program: asks for name and age (wrapping age in `int()`), works out next year's age, then prints "Hi Rav, next year you turn 31" using an f-string.

Let's combine everything — ask, convert, calculate, and reply:

```
name = input("Your name? ")
age  = int(input("Your age? "))    # wrap in int() to do maths
next_age = age + 1
print(f"Hi {name}, next year you turn {next_age}")
# -> Hi Rav, next year you turn 31
```



This little program does the full loop: **speak** (ask), **listen** (input), **convert** (int), **calculate** (+1), **speak again** (f-string). Notice we wrapped `age` in `int()` so the `+ 1` maths works. That five-line pattern — ask, convert, compute, reply — is the skeleton of countless real programs.

Recap — the 20-second version

- `print(...)` **speaks** to the user; `input(...)` **listens** and hands back what they typed.
- **input()** always gives back **TEXT** — wrap it in `int(...)` or `float(...)` to get a number.
- `print("a", 30, "b")` — separate with **commas** to show several things (auto-spaced).
- Comments start with `#` — notes for humans, ignored by Python.
- Explain the **WHY** in comments, not the obvious.

Next up — Video 6: Booleans & Comparisons. Every decision a program makes starts with a yes/no question — "is the age over 18?", "did the password match?". Next we meet True/False, the comparison operators, and how to combine them with and/or/not. See you tomorrow.

